

HW1 – CS 671

Yash Dagade

September 18, 2025

(a) No Regularization

1. **Why optimizing plain 0–1 loss is bad.** Because without a sparsity/complexity penalty a tree can grow arbitrarily deep to perfectly memorize the training set (zero training error) and badly overfit.
2. **Deepest valid tree.** Along any root-to-leaf path you can test each feature at most once, so you can have at most k splits. Since depth counts nodes (leaf included) and a tree with one leaf has depth 1, the deepest valid tree has depth

$$\boxed{k + 1}.$$

3. **Number of valid optimal trees for Table 1.** Dataset: $(1, 0) \rightarrow 0$ (twice), $(0, 1) \rightarrow 1$ (once). Zero training error is achievable.

- Depth-2 stumps:

(a) Root split on x_1 : $x_1=1 \mapsto 0$, $x_1=0 \mapsto 1$.

(b) Root split on x_2 : $x_2=1 \mapsto 1$, $x_2=0 \mapsto 0$.

$\Rightarrow$ 2 trees.

- Depth-3 extensions (root x_1 , mirror for x_2):

– Split only one child on x_2 : 2 choices of child $\times$ 2 free labelings = 4 trees.

– Split both children on x_2 : 2 leaves fixed, 2 free $\Rightarrow 2^2 = 4$ trees.

$\Rightarrow$ 8 trees per root. Since root can be x_1 or x_2 , total 16.

Total valid optimal trees:

$$\boxed{18}.$$

4. **Unique logical expressions among these trees.** Constraints: $f(1, 0) = 0$, $f(0, 1) = 1$. Unseen points $(0, 0)$ and $(1, 1)$ are free. That yields $2 \times 2 = 4$ distinct Boolean functions:

$$(0, 0) \rightarrow 0, (1, 1) \rightarrow 0 : f = \neg x_1 \wedge x_2,$$

$$(0, 0) \rightarrow 0, (1, 1) \rightarrow 1 : f = x_2,$$

$$(0, 0) \rightarrow 1, (1, 1) \rightarrow 0 : f = \neg x_1,$$

$$(0, 0) \rightarrow 1, (1, 1) \rightarrow 1 : f = \neg x_1 \vee x_2.$$

So the number is

$$\boxed{4}.$$

(b) Soft Regularization

We define

$$R(f, X, y|\lambda) = L(f, X, y) + \lambda s(f),$$

where $s(f)$ is the number of leaves.

1. **Largest possible R for one-leaf tree.** A one-leaf tree (constant predictor) can always choose the majority class, giving $L \leq \frac{1}{2}$. Thus

$$R \leq \frac{1}{2} + \lambda.$$

2. **Accuracy needed for deeper prefix-refinement.** Refining f_d into f_{d+k} adds at least k leaves, so $s(f_{d+k}) \geq s(f_d) + k$. For R to improve:

$$L(f_{d+k}) + \lambda(s(f_d) + k) < L(f_d) + \lambda s(f_d),$$

hence

$$L(f_{d+k}) \leq L(f_d) - \lambda k.$$

So the deeper tree must reduce misclassification by at least

$$\boxed{\lambda k}.$$

3. **Depth guarantee for suboptimality.** Any depth- d tree has at least $s \geq d$ leaves, hence $R \geq \lambda d$. A depth-1 tree has $R \leq \frac{1}{2} + \lambda$. Therefore any depth d with

$$\lambda d > \frac{1}{2} + \lambda \iff d > 1 + \frac{1}{2\lambda}$$

is always worse than some one-leaf tree. Thus the smallest such depth is

$$\boxed{d^*(\lambda) = \left\lfloor 1 + \frac{1}{2\lambda} \right\rfloor + 1}.$$

(If this exceeds the maximum feasible depth $k + 1$, the statement is vacuously true.)

4. **Number of optimal trees for Table 1, $0 < \lambda < \frac{1}{2}$.**

Candidate objectives:

- One-leaf majority (predict 0): $R = \frac{1}{3} + \lambda$.
- Zero-loss stumps (2 leaves): $R = 2\lambda$.
- Deeper zero-loss trees (3 or 4 leaves): $R = 3\lambda$ or 4λ (never optimal for $\lambda > 0$).

Compare 2λ vs $\frac{1}{3} + \lambda$:

- $0 < \lambda < \frac{1}{3}$: stumps win $\Rightarrow \boxed{2}$ optimal trees.
- $\lambda = \frac{1}{3}$: tie $\Rightarrow 2$ stumps + 1 one-leaf $\Rightarrow \boxed{3}$ optimal trees.
- $\frac{1}{3} < \lambda < \frac{1}{2}$: one-leaf wins $\Rightarrow \boxed{1}$ optimal tree.